



```
}

In loop.c:

bool GOMP_loop_dynamic_start (long start, long end, long incr,
                             long chunk_size, long *istart, long *iend)

{
    return gomp_loop_dynamic_start (start, end, incr, chunk_size, istart, iend);
}
```

```
In loop.c:

static bool gomp_loop_dynamic_start (long start, long end, long incr,
                                     long chunk_size, long *istart, long *iend)

{
    struct gomp_thread *thr = gomp_thread ();
    bool ret;

    if (gomp_work_share_start (false))
    {
        gomp_loop_init (thr->ts.work_share, start, end, incr,
                        GFS_DYNAMIC, chunk_size);
        gomp_work_share_init_done ();
    }

#ifdef HAVE_SYNC_BUILTINS
    ret = gomp_iter_dynamic_next (istart, iend);
#else
    gomp_mutex_lock (&thr->ts.work_share->lock);
    ret = gomp_iter_dynamic_next_locked (istart, iend);
    gomp_mutex_unlock (&thr->ts.work_share->lock);
#endif

    return ret;
}
```

```
// Chunk execution.

// Call GOMP_loop_dynamic_next.
```

```
In loop.c:

bool GOMP_loop_dynamic_next (long *istart, long *iend)
{
    return gomp_loop_dynamic_next (istart, iend);
}
```

```
In loop.c:

static bool gomp_loop_dynamic_next (long *istart, long *iend)
{
    bool ret;
```

```
    gomp_parallel_loop_start (fn, data, num_threads, start, end, incr,
                              GFS_DYNAMIC, chunk_size, 0);
}

In loop.c:

static void gomp_parallel_loop_start (void (*fn) (void *), void *data,
                                     unsigned num_threads, long start, long end,
                                     long incr, enum gomp_schedule_type sched,
                                     long chunk_size, unsigned int flags)

{

    struct gomp_team *team;

    num_threads = gomp_resolve_num_threads (num_threads, 0);
    team = gomp_new_team (num_threads);
    gomp_loop_init (&team->work_shares[0], start, end, incr, sched, chunk_size);
    gomp_team_start (fn, data, num_threads, flags, team);
}
```

```
In loop.c:

bool GOMP_loop_dynamic_next (long *istart, long *iend)
{

    return gomp_loop_dynamic_next (istart, iend);
}
```

```
In loop.c:

static bool gomp_loop_dynamic_next (long *istart, long *iend){

    bool ret;
#ifdef HAVE_SYNC_BUILTINS
    ret = gomp_iter_dynamic_next (istart, iend);
#else
    struct gomp_thread *thr = gomp_thread ();
    gomp_mutex_lock (&thr->ts.work_share->lock);
    ret = gomp_iter_dynamic_next_locked (istart, iend);
    gomp_mutex_unlock (&thr->ts.work_share->lock);
#endif

    return ret;
}
```

```
// Chunk execution.

// Call GOMP_loop_dynamic_next.
```

```
In loop.c:

bool GOMP_loop_dynamic_next (long *istart, long *iend)
{

    return gomp_loop_dynamic_next (istart, iend);
}
```

```
In loop.c:

static bool gomp_loop_dynamic_next (long *istart, long *iend)
```

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>#ifndef HAVE_SYNC_BUILTINS      ret = gomp_iter_dynamic_next (istart, iend); #else     <b>struct</b> gomp_thread *thr = gomp_thread ();     gomp_mutex_lock (&amp;thr-&gt;ts.work_share-&gt;lock);     ret = gomp_iter_dynamic_next_locked (istart, iend);     gomp_mutex_unlock (&amp;thr-&gt;ts.work_share-&gt;lock); #endif      <b>return</b> ret; }  // Volta para a execução do chunk ou sai.  In loop.c:  void  GOMP_loop_end_nowait (<b>void</b>) {     gomp_work_share_end_nowait (); }</pre> | <pre>{     bool ret;  #ifdef HAVE_SYNC_BUILTINS      ret = gomp_iter_dynamic_next (istart, iend); #else     <b>struct</b> gomp_thread *thr = gomp_thread ();     gomp_mutex_lock (&amp;thr-&gt;ts.work_share-&gt;lock);     ret = gomp_iter_dynamic_next_locked (istart, iend);     gomp_mutex_unlock (&amp;thr-&gt;ts.work_share-&gt;lock); #endif      <b>return</b> ret; }  // Volta para a execução do chunk ou sai.  In loop.c:  void  GOMP_loop_end_nowait (<b>void</b>) {     gomp_work_share_end_nowait (); }</pre> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

| Application           |            | Call function/Operation         | First format<br>(chunk is a variable or one expression)                                                                                                                                                                                                                                                                                       | Second format<br>(chunk is a value or a constant)                                                                                                                                                                                                                                                                                             |
|-----------------------|------------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Parallel Region Start |            |                                 | GOMP_parallel_start<br>(main._omp_fn.0, &.omp_data_o.1, 4);                                                                                                                                                                                                                                                                                   | GOMP_parallel_loop_dynamic_start<br>(main._omp_fn.0, &.omp_data_o.1, 4, 0, 1025, 1, 4);                                                                                                                                                                                                                                                       |
|                       |            |                                 | >>> Create and start the team.<br>gomp_team_start(...)                                                                                                                                                                                                                                                                                        | >>> Create and start the team, with Initialization of loop.<br>gomp_new_team(...), gomp_loop_init(...), gomp_team_start(...)                                                                                                                                                                                                                  |
| Call function         |            | main._omp_fn.0 (&.omp_data_o.1) | Function Context                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                               |
|                       | Loop Start | Initial Chunk                   | GOMP_loop_dynamic_start<br>(0, 1025, 1, D.1753, &.istart0.2, &.iend0.3);                                                                                                                                                                                                                                                                      | GOMP_loop_dynamic_next<br>(&.istart0.2, &.iend0.3);                                                                                                                                                                                                                                                                                           |
|                       |            |                                 | >>> Create and start work share: Loop initialization.<br>>>> Get the first set of iterations.<br>gomp_loop_init(...), gomp_iter_dynamic_next_*(...)                                                                                                                                                                                           | >>> Get the first set of iterations.<br>gomp_iter_dynamic_next_*(...)                                                                                                                                                                                                                                                                         |
|                       |            | Execution                       | <bb 9>:<br>.istart0.4 = .istart0.2;<br>i = (int) .istart0.4;<br>.iend0.5 = .iend0.3;<br>D.1760 = (int) .iend0.5;<br><br><bb 4>:<br>D.1761 = (long unsigned int) i;<br>D.1762 = D.1761 * 4;<br>D.1763 = .omp_data_i->a;<br>D.1764 = D.1763 + D.1762;<br>*D.1764 = 1;<br>i = i + 1;<br>if (i < D.1760)<br>goto <bb 4>;<br>else<br>goto <bb 10>; | <bb 9>:<br>.istart0.4 = .istart0.2;<br>i = (int) .istart0.4;<br>.iend0.5 = .iend0.3;<br>D.1760 = (int) .iend0.5;<br><br><bb 4>:<br>D.1761 = (long unsigned int) i;<br>D.1762 = D.1761 * 4;<br>D.1763 = .omp_data_i->a;<br>D.1764 = D.1763 + D.1762;<br>*D.1764 = 1;<br>i = i + 1;<br>if (i < D.1760)<br>goto <bb 4>;<br>else<br>goto <bb 10>; |
|                       |            | Next Chunk                      | GOMP_loop_dynamic_next (&.istart0.2, &.iend0.3);                                                                                                                                                                                                                                                                                              | GOMP_loop_dynamic_next (&.istart0.2, &.iend0.3);                                                                                                                                                                                                                                                                                              |
|                       |            |                                 | >>> Get the next set of iterations.<br>gomp_iter_dynamic_next_*(...)                                                                                                                                                                                                                                                                          | >>> Get the next set of iterations.<br>gomp_iter_dynamic_next_*(...)                                                                                                                                                                                                                                                                          |
|                       | Loop End   | Return of Function Call         | GOMP_loop_end_nowait ();                                                                                                                                                                                                                                                                                                                      | GOMP_loop_end_nowait ();                                                                                                                                                                                                                                                                                                                      |
|                       |            |                                 | >>> Finish the work share.<br>gomp_work_share_end_nowait()                                                                                                                                                                                                                                                                                    | >>> Finish the work share.<br>gomp_work_share_end_nowait()                                                                                                                                                                                                                                                                                    |
| Parallel Region End   |            |                                 | GOMP_parallel_end ();                                                                                                                                                                                                                                                                                                                         | GOMP_parallel_end ();                                                                                                                                                                                                                                                                                                                         |
|                       |            |                                 | >>> Finish the parallel region.<br>gomp_team_end ();                                                                                                                                                                                                                                                                                          | >>> Finish the parallel region.<br>gomp_team_end ();                                                                                                                                                                                                                                                                                          |

| Application              |  | Call function/Operation         | Second Format<br>(chunk or loop upper bound is a variable or one expression)                                                                                                                                                                                                                                                                  |  |
|--------------------------|--|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| Parallel Region<br>Start |  |                                 | GOMP_parallel_start (main._omp_fn.0, &.omp_data_o.1, 4);                                                                                                                                                                                                                                                                                      |  |
|                          |  |                                 | >>> <i>Create and start the team.</i><br>gomp_team_start(...)                                                                                                                                                                                                                                                                                 |  |
| Call function            |  | main._omp_fn.0 (&.omp_data_o.1) | Function Context                                                                                                                                                                                                                                                                                                                              |  |
| Loop Start               |  | Initial Chunk                   | GOMP_loop_dynamic_start (0, 1025, 1, D.1753, &.istart0.2, &.iend0.3);                                                                                                                                                                                                                                                                         |  |
|                          |  |                                 | >>> <i>Create and start work share: Loop initialization.</i><br>>>> <i>Get the first set of iterations.</i><br>gomp_loop_init(...), gomp_iter_dynamic_next_*(...)                                                                                                                                                                             |  |
|                          |  | Execution                       | <bb 9>:<br>.istart0.4 = .istart0.2;<br>i = (int) .istart0.4;<br>.iend0.5 = .iend0.3;<br>D.1760 = (int) .iend0.5;<br><br><bb 4>:<br>D.1761 = (long unsigned int) i;<br>D.1762 = D.1761 * 4;<br>D.1763 = .omp_data_i->a;<br>D.1764 = D.1763 + D.1762;<br>*D.1764 = 1;<br>i = i + 1;<br>if (i < D.1760)<br>goto <bb 4>;<br>else<br>goto <bb 10>; |  |
|                          |  | Next Chunk                      | GOMP_loop_dynamic_next (&.istart0.2, &.iend0.3);                                                                                                                                                                                                                                                                                              |  |
|                          |  |                                 | >>> <i>Get the next set of iterations.</i><br>gomp_iter_dynamic_next_*(...)                                                                                                                                                                                                                                                                   |  |
| Loop End                 |  | Return of Function Call         | GOMP_loop_end_nowait ();                                                                                                                                                                                                                                                                                                                      |  |
|                          |  |                                 | >>> <i>Finish the work share.</i><br>gomp_work_share_end_nowait()                                                                                                                                                                                                                                                                             |  |
| Parallel Region<br>End   |  |                                 | GOMP_parallel_end ();                                                                                                                                                                                                                                                                                                                         |  |
|                          |  |                                 | >>> <i>Finish the parallel region.</i><br>gomp_team_end ();                                                                                                                                                                                                                                                                                   |  |

| Application              | Call function/Operation        | First Format<br>(chunk or loop upper bound is a numerical value or a constant)                                                                                                                                                                                                                                                                                                                             |  |
|--------------------------|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| Parallel Region<br>Start |                                | GOMP_parallel_loop_dynamic_start (main._omp_fn.0, &.omp_data_o.1, 4, 0, 1025, 1, 4);                                                                                                                                                                                                                                                                                                                       |  |
|                          |                                | >>> <i>Create and start the team, with Initialization of loop.</i><br>gomp_new_team(...), gomp_loop_init(...), gomp_team_start(...)                                                                                                                                                                                                                                                                        |  |
| Call function            | main._omp_fn.0(&.omp_data_o.1) |                                                                                                                                                                                                                                                                                                                                                                                                            |  |
| Loop Start               | Initial Chunk                  | GOMP_loop_dynamic_next (&.istart0.2, &.iend0.3);                                                                                                                                                                                                                                                                                                                                                           |  |
|                          |                                | >>> <i>Get the first set of iterations.</i><br>gomp_iter_dynamic_next_*(...)                                                                                                                                                                                                                                                                                                                               |  |
|                          | Execution                      | <pre>&lt;bb 9&gt;:     .istart0.4 = .istart0.2;     i = (int) .istart0.4;     .iend0.5 = .iend0.3;     D.1760 = (int) .iend0.5;      &lt;bb 4&gt;:     D.1761 = (long unsigned int) i;     D.1762 = D.1761 * 4;     D.1763 = .omp_data_i-&gt;a;     D.1764 = D.1763 + D.1762;     *D.1764 = 1;     i = i + 1;     if (i &lt; D.1760)         goto &lt;bb 4&gt;;     else         goto &lt;bb 10&gt;;</pre> |  |
|                          | Next Chunk                     | GOMP_loop_dynamic_next (&.istart0.2, &.iend0.3);                                                                                                                                                                                                                                                                                                                                                           |  |
|                          |                                | >>> <i>Get the next set of iterations.</i><br>gomp_iter_dynamic_next_*(...)                                                                                                                                                                                                                                                                                                                                |  |
| Loop End                 | Return of Function Call        | GOMP_loop_end_nowait ();                                                                                                                                                                                                                                                                                                                                                                                   |  |
|                          |                                | >>> <i>Finish the work share.</i><br>gomp_work_share_end_nowait()                                                                                                                                                                                                                                                                                                                                          |  |
| Parallel Region<br>End   |                                | GOMP_parallel_end ();                                                                                                                                                                                                                                                                                                                                                                                      |  |
|                          |                                | >>> <i>Finish the parallel region.</i><br>gomp_team_end ();                                                                                                                                                                                                                                                                                                                                                |  |